

PART IV — FROM RAG TO AGENTIC RAG

Chapter 21

Answer-Relevance Verification (Separate From Groundedness)

“A fully grounded answer to the wrong question is still the wrong answer.”

Chapter 21 — Answer-Relevance Verification (Separate From Groundedness)

Memora's own `GROUNDING\_PROMPT` already gestures at this — its `OFF\_TOPIC` verdict and its original "relevance" criterion both exist because groundedness alone was never going to be sufficient — but relevance and groundedness are folded into one judge call rather than verified as two separable concerns. This chapter takes that folded-together check apart, using patterns this project has already proven elsewhere: a cheap filter before an expensive one, a rubric with a reason attached to every verdict, and thresholds tuned from real interaction logs instead of guessed.

21.1 Three distinct failure modes

Definition — Answer relevance

Whether a generated answer actually addresses the question that was asked, independent of whether its claims are individually true or traceable to evidence — a property an answer can fail even when every sentence in it is accurate.

Failure mode	What it looks like
Topic drift	The answer wanders from the question's actual subject into adjacent, better-evidenced territory
Question-type mismatch	A "why" question answered with a "what"; a yes/no question answered with a description
Hallucination	The answer invents content — the one mode groundedness already catches on its own

Only the third mode is groundedness's job. Topic drift and question-type mismatch can both happen with every claim in the answer perfectly traceable to real retrieved text — the retriever found real evidence, the generator wrote real sentences from it, and none of it actually answers what was asked. A judge with only one axis has no vocabulary to distinguish "this is wrong" from "this is true but not what you asked."

Topic drift is the retrieval-shaped version of this failure: a query with two plausible readings — Chapter 12's example of "ASD" meaning either autism spectrum disorder or adjustable-speed-drive is the recurring case throughout this project's own corpus — retrieves real, well-grounded chunks about the wrong reading entirely, and an answer built faithfully from those chunks drifts with them. Question-type mismatch is a narrower, almost mechanical failure: a "why does X happen" question answered with a well-grounded description of *what* X is, never touching the causal mechanism the question actually asked about. Neither failure requires the model to invent anything — both can happen with a generator behaving exactly as instructed, working from exactly the evidence it was given.

21.2 Why pure cosine similarity is the wrong tool on its own

Embedding the question and the answer and comparing them with cosine similarity is the obvious first instinct, and it fails in both directions at once. A fluent, on-topic-*sounding* answer that shares heavy vocabulary with the question scores a high cosine similarity regardless of whether its actual content is correct — the embedding captures topical proximity, not factual correctness. A short, correct answer that happens to use different words than the question — a common outcome of good paraphrase — can score a *lower* cosine similarity than a wordier, wrong one.

Two ways cosine similarity alone gets it wrong

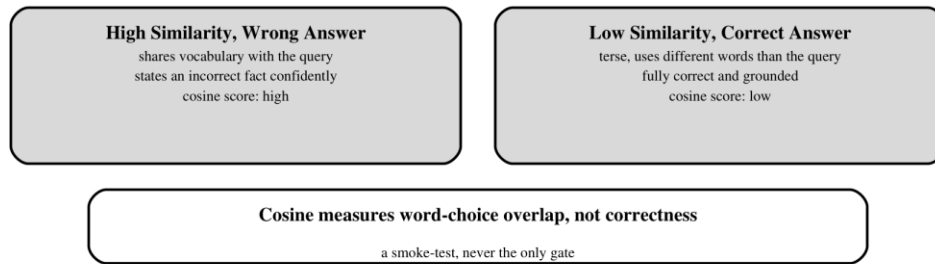


Figure 21.1 — Cosine similarity between query and answer measures vocabulary overlap, which correlates with but does not equal relevance.

This is the same shallow-signal problem Chapter 13.8 and Chapter 20.5 already named from two other angles — a cheap, retrieval-derived number is a real, useful signal, but only for what it actually measures. Cosine similarity between query and answer measures word-choice proximity. Relevance is a judgment about content. Treating the first as a substitute for the second fails exactly the two ways Figure 21.1 shows, in both directions.

Both failure directions are worth sitting with because they push a naive threshold in opposite ways. Set the threshold high enough to catch every high-similarity-wrong-answer case, and the low-similarity-correct-answer case starts failing the smoke-test too, rejecting good answers for the crime of being concise. Set it low enough to let every good terse answer through, and confidently wrong, vocabulary-matched answers start passing through unchallenged. There is no single threshold that solves both directions at once with cosine similarity alone — which is precisely the argument for demoting it to a smoke-test rather than asking it to be the whole gate.

21.3 The two-stage gate — similarity smoke-test plus LLM judge

The fix is not to abandon cosine similarity — it is to demote it to what it is actually good at: a cheap first filter that resolves the easy cases and defers only the ambiguous ones to an LLM call. This is the identical shape as Chapter 20B's layered repair pipeline and ADR-050's pre-retrieval cosine filter — try the inexpensive signal first, escalate only when it can't resolve the question on its own.

A cheap filter earns the right to an expensive one

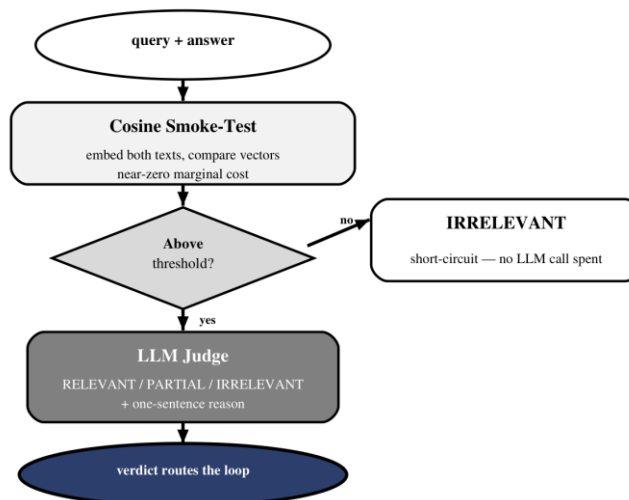


Figure 21.2 — A smoke-test below threshold short-circuits to IRRELEVANT; only the ambiguous middle ever reaches the judge.

```
def relevance_gate(query, answer, embed, judge_llm, smoke_threshold=0.2):
    similarity = cosine(embed(query), embed(answer))
    if similarity < smoke_threshold:
        return {"verdict": "IRRELEVANT", "reason": "below similarity smoke-
test floor"}
    return relevance_judge(query, answer, judge_llm)    # Section 21.4
```

The threshold in Figure 21.2 is set deliberately low — this stage exists to catch only the confidently, unambiguously off-topic cases cheaply, not to make the real relevance call. Anything above it, including plenty of genuinely irrelevant answers that merely share some vocabulary with the question, still reaches the judge. A smoke-test that tries to do the judge's job ends up with the judge's failure modes at the smoke-test's level of precision.

21.4 Designing the relevance rubric

Every judge already built in this project shares one shape: a constrained verdict enum plus a mandatory one-sentence reason, never a bare label alone. A relevance judge follows the identical pattern `RetrievalJudgeSchema` (Chapter 20B.6) already established, applied to a different question.

```
class RelevanceJudgeSchema(_BaseStrict):
    verdict: Literal["RELEVANT", "PARTIAL", "IRRELEVANT"]
    reason: str

RELEVANCE_JUDGE_PROMPT = """You are judging whether an ANSWER addresses a
QUESTION —
not whether the answer is factually correct or well-grounded, only whether
it
responds to what was actually asked.

QUESTION: {query}
ANSWER: {answer}

RELEVANT: directly and substantially addresses the question.
PARTIAL: addresses part of the question, or addresses it only tangentially.
IRRELEVANT: does not address the question asked, regardless of its own
accuracy.

Return ONLY a JSON object:  {"verdict": "...", "reason": "<one
sentence>"}"""
```

Common pitfall — Letting the relevance prompt drift into grounding

A relevance judge that starts asking "is this true" or "is this supported by evidence" has stopped measuring relevance and started re-measuring Chapter 20's axis, at greater cost and with two judges now disagreeing about the same thing. The prompt above states its exclusion explicitly — **not whether the answer is factually correct** — for exactly this reason. Keep the two judges' questions as narrow and non-overlapping as their names promise.

21.5 Tuning the cosine threshold from real interaction data

A smoke-test threshold picked by intuition is a threshold picked wrong in one direction or the other. This project has already run the correct methodology twice — ADR-050 derived its `0.95` inter-variant dedup threshold from 775 real variant pairs pulled from `interactions.jsonl`, and the `DOCUMENTS\_MIN\_SIMILARITY`/`LEARNED\_QA\_MIN\_SIMILARITY` retrieval floors were revised from `0.50` to `0.53`/`0.57` after an A/B comparison across real debug logs measured LLM-call counts and verdict quality at each candidate value directly, rather than picking one and hoping.

- Pull a sample of real (query, answer, human-or-judge-labeled relevance) triples from your own interaction log.
- Compute cosine similarity for every pair and plot the distribution split by label.
- Pick the smoke-test floor from the low tail of the RELEVANT distribution, not its center — the smoke-test's job is to catch only confident IRRELEVANT cases, not to separate RELEVANT from PARTIAL.
- Re-derive the threshold whenever the embedding model changes — Chapter 11.1's warning about comparing vectors from two different models applies here exactly as it did to retrieval.

The discipline matters more than the specific number. Both of this project's real precedents — the 0.95 dedup threshold and the 0.53/0.57 retrieval floors — were revised at least once after the first empirically-derived value turned out to be slightly wrong in production, not accepted permanently on the first measurement. Treat a smoke-test threshold the same way: a starting point to monitor and revisit as real answer traffic accumulates, not a constant to set once in `config.py` and forget.

21.6 What to do when relevance fails — retry or admit the gap

An `IRRELEVANT` or `PARTIAL` verdict deserves the same budget-aware treatment Chapter 18.5 and Chapter 20.4 already built for groundedness failures — not an automatic retry, and not silent acceptance. If retrieval budget remains, the retry message should say specifically that the **question wasn't addressed**, not that the answer was ungrounded — a model told the wrong thing about its own failure will often fix the wrong thing.

```
if relevance_verdict == "IRRELEVANT" and budget_remaining:
    retry_message = (
        "Your answer did not address the question asked. "
        "Re-read the question and answer it directly, using the retrieved "
        "context."
    )
elif relevance_verdict == "IRRELEVANT":
    return honest_gap_answer(query) # Chapter 20.6's graceful
    degradation, same principle
```

A `'PARTIAL'` verdict is the more common real-world case and deserves its own branch rather than collapsing into the same handling as `'IRRELEVANT'` — a retry message for a partially-relevant answer should ask the model to **complete** its coverage, not start over, the same distinction Chapter 20.7 drew between `'OVERCLAIMED'` and `'PARTIALLY_FABRICATED'` for groundedness. Two verdicts that both mean "not good enough yet" can still call for two different next actions.

Admitting the gap is not a failure state to minimize — it is the correct outcome once budget is genuinely exhausted, and Chapter 20.6 already made the case for why a canned, honest refusal beats a padded answer built from whatever evidence happened to be on hand. The same logic applies here with one addition specific to relevance: a gap admission triggered by an `'IRRELEVANT'` verdict should say so plainly — "the available information does not directly address this question" reads very differently from a generic "I don't know," and gives the user an accurate reason to reformulate their own question rather than assume the knowledge base is simply empty.

21.7 Keeping relevance separate from groundedness in the agent loop

Memora's actual `'GroundingJudgeSchema'` bundles both concerns into one verdict enum — `'OFF_TOPIC'` sits alongside `'PARTIALLY_FABRICATED'` and `'OVERCLAIMED'` in a single field, decided by a single judge call. That design is a defensible cost tradeoff, not an oversight: one LLM call is cheaper than two, and Chapter 20's judge already asks the model to consider relevance as one of its rules. The architectural point this chapter makes is not that the call must be split — it is that the **concepts** must stay separable in how failures are reasoned about and routed, whether or not they are separable in how many API calls it takes to check them.

Chapter 20.8 named exactly the risk of collapsing distinct failure axes into one verdict: a judge that can only say pass or fail has no way to tell a caller **which** rule actually broke, and a retry strategy built on a single collapsed signal ends up guessing at a fix instead of targeting one. `'OFF_TOPIC'` as its own named value — not folded silently into a generic `'FAILED'` — is what makes it possible to route a topic-drift failure toward "try genuinely different query angles" and a fabrication failure toward "retry with stricter grounding constraints," even from inside one judge call. The lesson generalizes past this one schema: whenever a verdict enum grows past two values, ask whether each value routes somewhere different — if it doesn't, the distinction is decoration, and if it does, the distinction is load-bearing and deserves exactly the visibility `'OFF_TOPIC'` already has here.

There is a real cost argument for splitting the call anyway, worth naming even though Memora itself did not make that choice: a combined judge asks one model to hold two different rubrics in mind simultaneously, and Chapter 14.3 already flagged the general tension between a judge's output-format discipline and the reasoning depth a harder question deserves. A relevance-only judge and a groundedness-only judge each have a narrower, more mechanically checkable job — closer to Chapter 20B.6's schema-per-concern discipline than a single verdict enum straining to cover both axes. Whether that extra clarity is worth a second LLM call is a real tradeoff a project should make deliberately, the same way Chapter 13B.13's tiered SLM/LLM architecture made deliberate tradeoffs between judgment quality and call cost elsewhere in this pipeline — not a default either direction should be assumed without weighing it.

Groundedness answers "is this true, given the evidence." Relevance answers "is this an answer to the question." A pipeline needs both judgments, needs to know which one it is making at any given moment, and needs to route a failure of one differently from a failure of the other — whether that means two judges or one judge with a rich enough verdict space to keep the distinction alive.